WSAN Assignment 4

1. Using the examples given in the slides, write a code to create 5 thread of the function vMyThread(). This function should print a welcome message along with the thread id (pass the thread id to vMyThread() when creating the threads). Run your code for several times. What pattern do you see in the output numbers?

After running the code several times, the pattern seems to be consistent. The first two threads are created, and then it alternates between the threads (in order) exiting, and a new thread being created. The final two lines are the last two threads, 3 and 4, exiting.

2. Using the usleep() from unistd.h API, add a 1000us delay to the for loop of the main function and rerun the code for several times. How do you justify the change in the output?

With this version of the code, every thread is created, and then finishes, before the next thread is created. This is because the main thread is suspended by usleep while the new pthread that was just created continues its execution; it finishes before the main thread wakes up.

3. Comment the usleep() Define an int counter variable outside of main() and vMyThread() and initialize it to zero. Next, update the vMyThread() function and add a for loop to increase the counter variable and print a message, that includes the thread id and the counter variable. Using a for loop, repeat this operation for 5 times. To mimic a delay from running other commands and to see the prints, add a 1 second delay between the counter increase line and the print line. Run the code for several times. What pattern do you see in the printed thread id and the counter?

There's very little pattern. Each second, the threads execute in a fairly random order (e.g. 1, 4, 2, 0, 3) there's very little consistency in the value reported within the different "cycles" (e.g. thread 0, 1, and 2 report counter as 15, thread 3 and 4 report it as 17), although it does appear that the value of counter always ends up at 25.

4. Define a pthread_mutex_t outside of main() and vMyThread(). Using the mutex, lock and unlock our critical section (including the counter increase, the delay, and the print function). Run the code again. What pattern do you see in the printed thread id and the counter?

This makes the increases in the counter predictable, but at the cost of significantly decreasing the parallelism. The threads run almost entirely sequentially; thread 0 runs all five iterations of its loop before ceding to thread 1, which runs all five iterations. I did have a couple of runs where, at the end, threads 4 and 2 wouldn't run in their entirety (e.g. thread 4 ran iterations 0-2 and then thread 2 ran 0 and 1, then back to 4). However, for the most part, each thread runs its full loop, increasing the counter five times, before another thread can take over.

5. Create another delay of 10ms after unlocking the mutex. What changes do you see in the output?

This time, each thread runs one iteration of its loop, increasing the counter; once each thread has taken a turn, it restarts at thread 0. The threads run reliably in numeric order and the printed value of the counter is always what would be expected.